



LUMINA PRO

Application Kanban de gestion de tâches

SYRINE BEN HASSINE

SOMMAIRE

1 Présentation personnelle & reconversion

2 Pourquoi ce projet ?

3 État du marché du Kanban

4 User stories

5 Fonctionnalités & cahier des charges

6 Projet solo — justification

7 Planning

8 Choix des technologies

9 Architecture applicative

10 Wireframes & maquettes

11 Moodboard & charte graphique

12 Responsive

13 Base de données — MCD/MLD/MPD

14 Démo Kanban

15 Intégration HTML/CSS

16 Swagger

17 SQL — Exemples CRUD

18 Liste des routes

19 Authentification JWT

20 Sécurité

21 Accessibilité, SEO & Lighthouse

22 Cahier de tests

23 Déploiement

24 Améliorations futures

25 Conclusion

PRÉSENTATION PERSONNELLE & RECONVERSION

Qui suis-je, et pourquoi ce changement de voie ?

Passionnée de mode depuis toute petite, je suis diplômée en stylisme-modélisme et j'ai obtenu une certification en informatique et bureautique en 2001. J'ai ensuite été gérante d'ateliers de couture, où je réalisais déjà mon stylisme assisté par ordinateur. La passion pour la technologie n'a jamais disparu depuis cette première certification : de nature très curieuse, j'ai toujours voulu comprendre la face cachée derrière les belles vitrines des sites web — comment elles sont vraiment conçues. En découvrant la formation de La Plateforme_, j'ai voulu la poursuivre pour concrétiser cette curiosité et avancer vers ce monde de la technologie qui façonne l'avenir.

Formation : DWWM — Développeuse Web et Web Mobile · La Plateforme_ · Promotion 2025/2026

Une reconversion est un atout, pas une faiblesse — elle montre de la motivation et de la maturité de choix.

POURQUOI CE PROJET ?

D'où vient l'idée de Lumina Pro ?

L'idée de départ : beaucoup d'équipes perdent du temps à suivre leurs tâches sur des supports dispersés (messages, tableurs, post-its). Lumina Pro répond à un besoin simple — **tout centraliser dans un seul tableau visuel**, sans la complexité d'un outil comme Jira.

Le nom et la signature du projet résument cette intention :

« Organisez vos idées avec clarté »

L'anecdote derrière le choix : pendant le projet de groupe MarsIA, nous avons utilisé Trello pour organiser le travail d'équipe — j'ai beaucoup aimé ses fonctionnalités et la facilité avec laquelle il permettait de suivre l'avancement de chacun. C'est cette expérience concrète qui m'a donné envie de comprendre et de recréer moi-même un outil Kanban, de A à Z.

ÉTAT DU MARCHÉ DU KANBAN

Des outils puissants, mais souvent lourds

TRELLO

Simple et populaire. Utilisé pendant ce projet pour le suivi des fonctionnalités à développer.

MONDAY.COM

Outil de gestion de projet tout-en-un, très visuel, mais coûteux et surdimensionné pour une petite équipe.

JIRA

Très complet pour les équipes agiles, mais complexe à configurer et coûteux pour une petite structure.

ASANA

Riche en fonctionnalités (rapports, automatisations), au prix d'une prise en main plus longue.

Positionnement de Lumina Pro : un Kanban resserré sur l'essentiel, pensé comme outil de certification — pas une alternative commerciale à ces solutions.
L'objectif : prouver la maîtrise de toute la chaîne (BDD → API → sécurité → interface).

🕒 Durée de réalisation : **3 mois** (mai → juillet 2026).

USER STORIES

Besoins exprimés par profil

EN TANT QU'UTILISATEUR, JE PEUX...

- **Authentification** : me connecter et m'inscrire de façon sécurisée
- **Tâches** : voir le tableau Kanban de l'équipe, créer et assigner des tâches, les déplacer entre colonnes, modifier ou supprimer mes propres tâches
- **Équipe** : consulter la liste des membres
- **Suivi** : consulter les statistiques du projet

EN TANT QU'ADMIN, JE PEUX FAIRE TOUT ÇA, PLUS...

- **Tâches** : supprimer n'importe quelle tâche de l'équipe, pas seulement les miennes
- **Colonnes** : créer, renommer et supprimer une colonne du tableau
- **Équipe** : voir et retirer un membre de l'équipe

FONCTIONNALITÉS & CAHIER DES CHARGES

Fonctionnalités retenues (MVP livré)

- Tableau Kanban, colonnes personnalisables (À faire, En cours, Terminé)
- CRUD complet des tâches
- Authentification sécurisée (JWT + bcrypt)
- Rôles admin / utilisateur
- Interface responsive (desktop et mobile)
- Documentation API Swagger

ADMIN

Gère les colonnes et les utilisateurs, peut supprimer n'importe quelle tâche

UTILISATEUR

Crée et déplace des tâches, ne peut supprimer que ses propres tâches assignées

PROJET SOLO

Pourquoi seule plutôt qu'en équipe ?

- ▶ Le référentiel DWWM autorise et évalue un projet individuel
- ▶ Travailler seule oblige à comprendre **toute la chaîne** (BDD, back-end, front-end, sécurité) plutôt qu'une seule partie
- ▶ Pas de dépendance à la disponibilité d'un binôme sur le temps de formation

En contrepartie : un périmètre volontairement resserré (MVP), pour livrer un projet complet et solide plutôt qu'un projet ambitieux mais inachevé.

PLANNING — DIAGRAMME DE GANTT

Reconstitué depuis l'historique Git réel (mai → juillet 2026)



Suivi du projet via **Trello** (tableau Kanban des fonctionnalités à développer) et **33 commits Git** qui retracent l'historique réel du développement. Planning reconstitué a posteriori à partir de ces commits plutôt que d'un GANTT prévisionnel théorique.

CHOIX DES TECHNOLOGIES

⚡ FRONT-END

- HTML5 / CSS3 / JS ES6+ vanilla
- Pas de framework : comprendre chaque ligne

🔧 BACK-END

- Node.js v24.14.0 + Express 5.2.1
- jsonwebtoken 9.0.3 + bcrypt 6.0.0 pour la sécurité
- dotenv 17.4.2 (secrets en .env), cors 2.8.6

🗄️ BASE DE DONNÉES

- **MySQL 8.0.45** — relationnel, adapté à des données liées (tâches ↔ utilisateurs ↔ colonnes)
- mysql2 3.22.2 comme driver Node

🔧 OUTILS

- Swagger UI (swagger-ui-express 5.0.1), Git/GitHub, XAMPP, VS Code

Justification du choix "vanilla" : objectif pédagogique — être capable d'expliquer chaque mécanisme (fetch, DOM, JWT) sans dépendre d'un framework qui ferait le travail à ma place.

ARCHITECTURE APPLICATIVE

Un MVC allégé



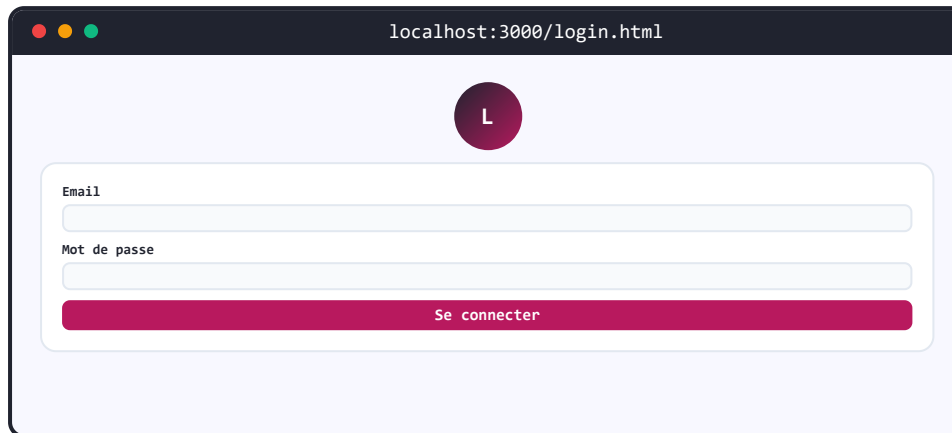
- **Vue** : pages HTML statiques (login.html, index.html) + modules JS qui manipulent le DOM
- **Contrôleur** : chaque fichier de backend/routes/ reçoit la requête HTTP et orchestre la réponse
- **Modèle** : pas de couche modèle séparée — chaque route exécute directement ses requêtes SQL via `req.db.query()`

Sur un projet solo de cette taille (4 tables), une séparation controller/model stricte aurait ajouté de l'indirection sans bénéfice réel. Cette simplification est identifiée comme première évolution si l'application devait grandir (voir roadmap).

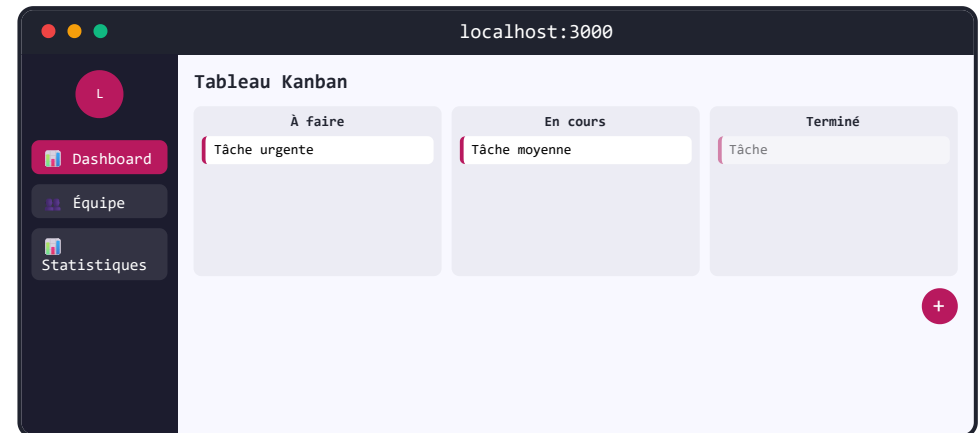
WIREFRAMES & MAQUETTES

Maquettes basse fidélité

login.html — Connexion / Inscription



index.html — Vue Kanban (dashboard protégé JWT)



Wireframes basse fidélité réalisés en amont pour valider la navigation avant tout travail graphique — détail complet (arborescence, tous les écrans) dans `arborescence_wireframes.html`.

MOODBOARD & INSPIRATIONS UX/UI

Référence visuelle ayant guidée la charte graphique

- ▶ Recherche de composants Kanban sur **Dribbble** pour la mise en page des colonnes et cartes de tâches
- ▶ **Palette rose/fuchsia** (#B8195E) : couleur de marque reconnaissable, associée à l'idée de créativité, tout en restant sobre sur le reste de l'interface
- ▶ **Sidebar sombre + contenu clair** : un schéma courant des outils SaaS de gestion — sépare clairement la navigation du contenu de travail, repère visuel immédiat, prise en main rapide
- ▶ **Peu de couleurs au total** : évite la surcharge visuelle, chaque couleur garde un sens précis — un code couleur unique et cohérent pour les priorités (rouge/ambre/vert)
- ▶ **Police système (Inter)** : pas de chargement externe, cohérence garantie sur toutes les machines
- ▶ **Ligne de flottaison respectée** : le tableau Kanban est visible entièrement sans avoir à scroller ; le **Call To Action** principal (bouton "+" pour créer une tâche) reste flottant et toujours visible en bas à droite, quelle que soit la position de scroll

Ces références (Dribbble + codes visuels SaaS) tiennent lieu de moodboard : pas de planche d'inspiration formelle constituée en amont, mais une recherche visuelle ciblée qui a directement nourri la charte graphique (couleurs, typographie, structure sidebar/contenu).

RESPONSIVE

Adaptation mobile avec CSS

Desktop

- ▶ Sidebar fixe à gauche
- ▶ Colonnes kanban côte à côte

Tablette ($\leq 900\text{px}$)

- ▶ Sidebar en tiroir (masquée par défaut), bouton hamburger ☰
- ▶ Colonnes kanban toujours côte à côte, largeur fixe (270px), scroll horizontal si besoin

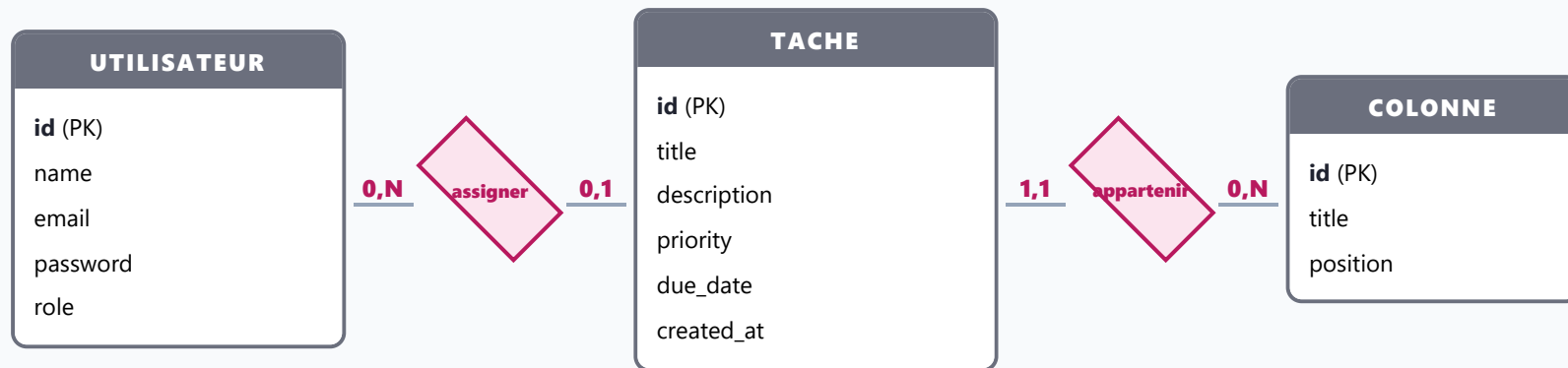
Mobile ($\leq 600\text{px}$)

- ▶ Sidebar en tiroir plein écran
- ▶ Colonnes empilées verticalement (scroll vertical, plus horizontal)

```
/* css/responsive.css */
@media (max-width: 900px) {
  .sidebar { left: -280px; }
  .sidebar.open { left: 0; }
  .kanban-column { min-width: 270px; width: 270px; }
}
@media (max-width: 600px) {
  .kanban-board { flex-direction: column; overflow-x: hidden; }
  .kanban-column { width: 100%; min-width: 100%; }
}
```

MCD — MODÈLE CONCEPTUEL DE DONNÉES

Entités, associations, cardinalités (méthode MERISE)



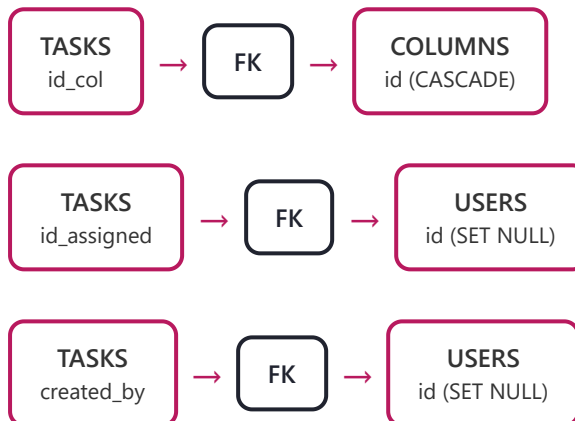
Lecture : Un UTILISATEUR peut être assigné à 0 à N tâches. Une TACHE peut être assignée à 0 ou 1 utilisateur (optionnel). Une TACHE appartient à *exactement 1* colonne. Une COLONNE peut contenir 0 à N tâches.

MLD — MODÈLE LOGIQUE DE DONNÉES

Traduction en tables, indépendante du SGBD

USERS		COLUMNS		TASKS	
Champ	Type	Champ	Type	Champ	Type
 id	PK	 id	PK	 id	PK
name	—	title	—	title	—
email	—	position	—	description	—
password	—			priority	—
role	—			due_date	—
team_status	—			created_at	—
				 id_assigned	FK → users.id (SET NULL)
				 id_col	FK → columns.id (CASCADE)
				 created_by	FK → users.id (SET NULL)

 = Clé primaire ·  = Clé étrangère · CASCADE = suppression en cascade · SET NULL = mise à NULL



MPD — MODÈLE PHYSIQUE DE DONNÉES

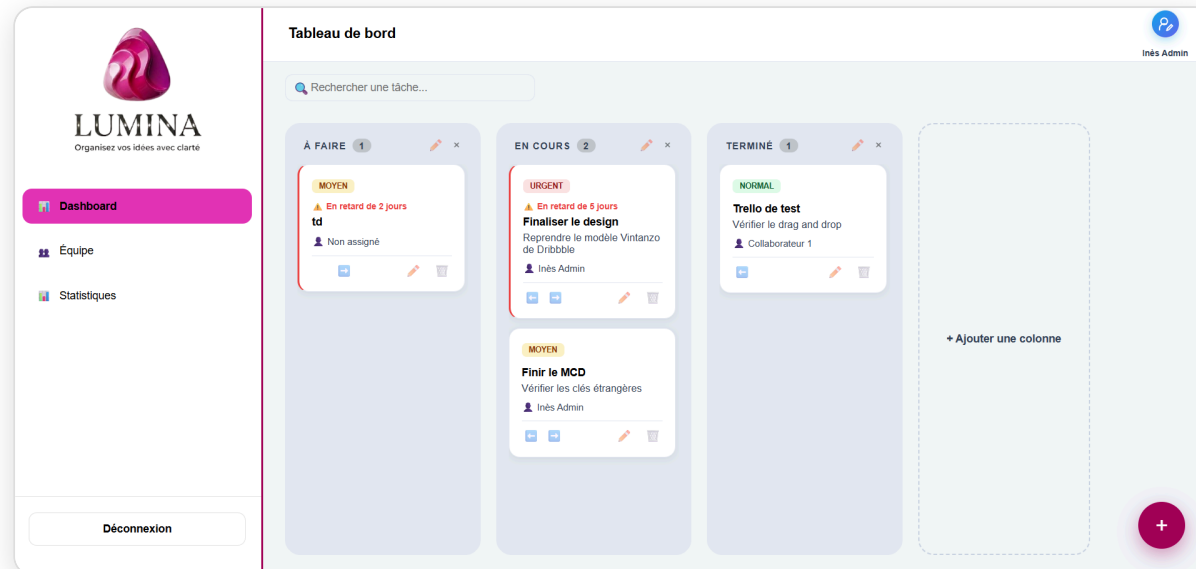
Implémentation MySQL (database/schema.sql)

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  role ENUM('admin','user') DEFAULT 'user',  
  team_status ENUM('pending','requested','member') DEFAULT 'pending'  
);  
  
CREATE TABLE columns (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  position INT DEFAULT 0  
);  
  
CREATE TABLE tasks (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL, description TEXT,  
  priority ENUM('high','medium','low') DEFAULT 'medium',  
  id_assigned INT, id_col INT, created_by INT, due_date DATE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_assigned) REFERENCES users(id) ON DELETE SET NULL,  
  FOREIGN KEY (id_col) REFERENCES columns(id) ON DELETE CASCADE,  
  FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL  
);
```

SGBD réel : **MySQL 8.0.45**. Le MPD ne contient pas de cardinalités — uniquement des types de colonnes et des contraintes.

KANBAN

Mes colonnes : À faire, En cours, Terminé



```
// frontend/js/kanban.js - renderBoard()
function renderBoard(columns, tasks) {
  board.innerHTML = columns.map(col => {
    const colTasks = tasks.filter(t => t.id_col === col.id);
    return `<div class="kanban-column"> ... ${colTasks.map(t => renderTask(t)).join('')} ... </div>`;
  }).join('');
}
```

INTÉGRATION FRONT HTML/CSS

Une base HTML/CSS structurée

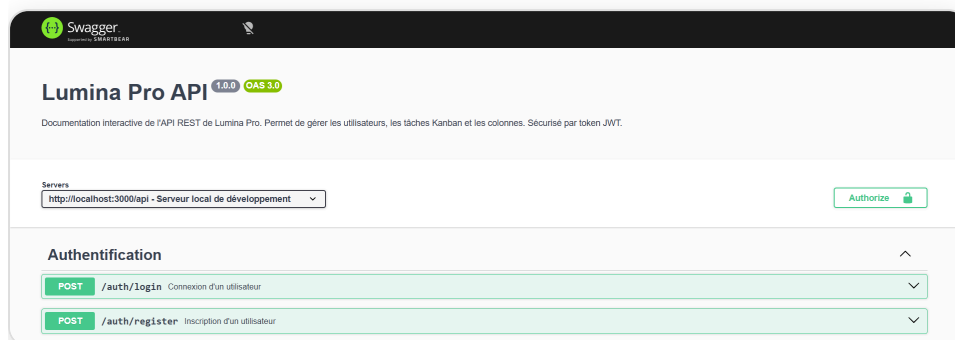
- ▶ HTML5 sémantique (header, nav, main)
- ▶ CSS3 découpé en **8 fichiers** par responsabilité (variables, layout, kanban, responsive, formulaires, composants...)
- ▶ Flexbox pour les mises en page, media queries pour le responsive
- ▶ Aucun framework CSS : chaque règle est écrite et comprise à la main

Rester connecté à chaque appel à l'API

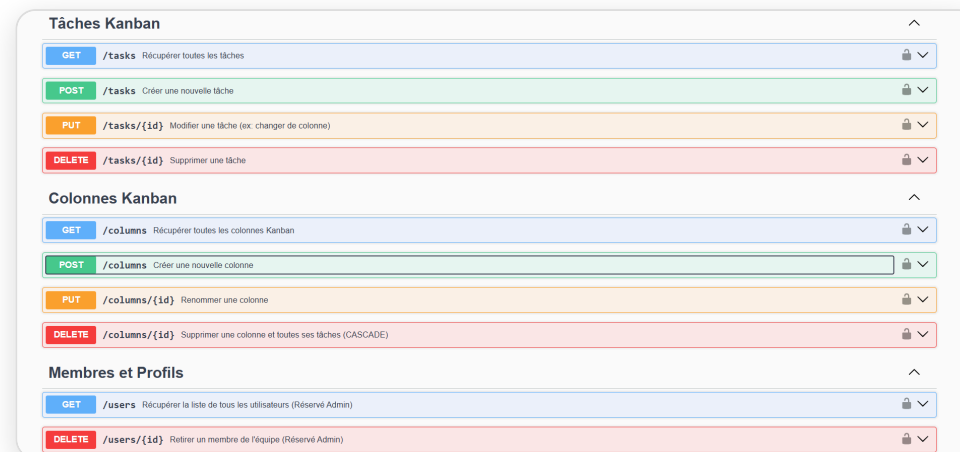
```
// frontend/js/api.js
async function authFetch(url, options = {}) {
  const token = localStorage.getItem('token');
  options.headers = { 'Authorization': `Bearer ${token}`, ...options.headers };
  const response = await fetch(url, options);
  if (response.status === 401) { localStorage.clear(); window.location.href = 'login.html'; }
  return response;
}
```

SWAGGER

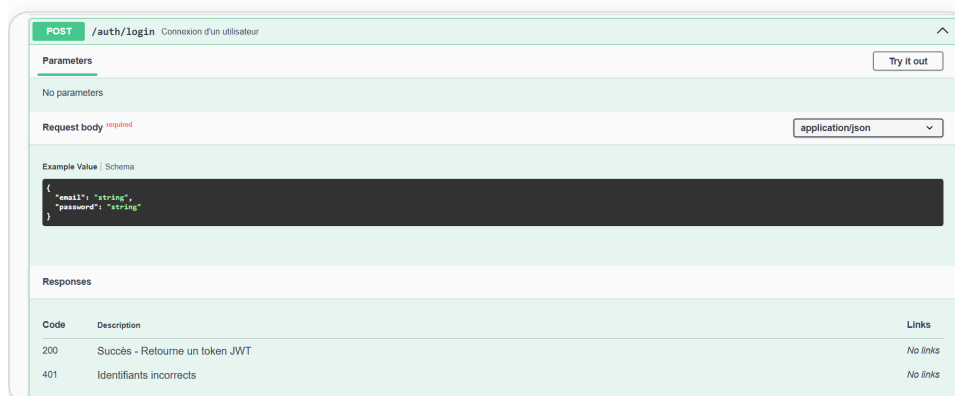
Documentation interactive générée automatiquement



Page d'accueil de **/api-docs**, générée automatiquement depuis swagger.json — titre, version de l'API, bouton d'autorisation JWT.



Les 18 routes regroupées par ressource (tâches, colonnes, membres...), chacune avec sa méthode HTTP.



Une route dépliée : schéma du corps de requête attendu et codes de réponse possibles, testable directement ici.

SQL — EXEMPLES CRUD

Un Create, un Read, un Update, un Delete

```
// 1. CREATE - routes/auth.js, à l'inscription
const sql = "INSERT INTO users (name, email, password) VALUES (?, ?, ?)";

// 2. READ - routes/tasks.js, liste des tâches
const sql = "SELECT * FROM tasks";

// 3. UPDATE - routes/tasks.js, construction dynamique selon les champs reçus
let sql = "UPDATE tasks SET "; if (title) sql += "title = ?, "; ... + " WHERE id = ?";

// 4. DELETE - la suppression d'une colonne entraîne celle de ses tâches
const sql = "DELETE FROM columns WHERE id = ?"; // ON DELETE CASCADE supprime ses tâches
```

Toutes les requêtes utilisent des placeholders ? (requêtes préparées) — jamais de concaténation directe de variables dans le SQL.

LISTE DES ROUTES DE MON API

API REST — server.js

```
app.use('/api/auth', require('./routes/auth'));
app.use('/api/tasks', verifyToken, require('./routes/tasks'));
app.use('/api/columns', verifyToken, require('./routes/columns'));
app.use('/api/users', verifyToken, require('./routes/users'));
```

Méthode	Route	Accès
POST	/api/auth/login, /register	Public
GET	/api/tasks, /api/columns, /api/users	JWT
POST / PUT	/api/tasks, /api/columns	JWT
DELETE	/api/tasks/:id	JWT — propriétaire ou admin
DELETE	/api/columns/:id, /api/users/:id	JWT (users/:id : admin)
PUT	/api/users/:id/role	JWT — admin
GET	/api/team/status, /api/team/requests, /api/logs/stats	JWT (requests : tout membre)
POST / PUT	/api/team/join, /api/team/requests/:id	JWT (requests/:id : tout membre)

Soit **18 routes** au total : 12 sur le cœur fonctionnel (auth/tasks/columns/users) + 6 sur l'intégration d'un nouveau membre à l'équipe, la promotion admin et les statistiques de suppression.

AUTHENTICATION JWT

JSON Web Token — flux complet



```
// middleware/security.js
function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader?.startsWith('Bearer '))
    return res.status(401).json({ error: 'Token manquant' });
  try {
    req.user = jwt.verify(authHeader.split(' ')[1], SECRET_KEY);
    next();
  } catch { res.status(403).json({ error: 'Token invalide' }); }
}
```

SÉCURITÉ



Injection SQL → requêtes préparées

Placeholders ? sur toutes les requêtes — jamais de concaténation directe.



XSS → escapeHTML()

Tout contenu utilisateur affiché dans le DOM est échappé avant insertion.



Mots de passe → bcrypt (10 rounds)

`bcrypt.hash(password, 10)` à l'inscription, `bcrypt.compare()` à la connexion — jamais stocké en clair.



CSRF / CORS / RGPD

Token JWT en header (jamais en cookie) ; CORS activé (à restreindre en prod) ; RGPD : collecte minimale (nom, email, mot de passe haché), jamais transmises à des tiers.



Suppression restreinte au propriétaire

Un utilisateur ne peut supprimer que ses propres tâches ; seul l'admin peut tout supprimer — vérifié côté serveur.

ACCESSIBILITÉ, SEO & Lighthouse

Accessibilité (RGAA / WCAG 2.1)

- ▶ `aria-label` sur les boutons icônes (sans texte visible)
- ▶ Navigation clavier complète, focus visible sur chaque élément interactif
- ▶ Contrastes vérifiés, `<label>` associé à chaque champ de formulaire
- ▶ `aria-live="polite"` sur les zones qui changent dynamiquement (liste des tâches)

SEO

- ▶ HTML5 sémantique (header, nav, main), un seul `<h1>` par page
- ▶ `<strong>` pour marquer les mots-clés importants dans le texte (le moteur de recherche leur accorde plus de poids qu'à du texte normal, sans changer le sens visuel — c'est du gras sémantique, pas juste stylistique)
- ▶ `<meta name="description">` sur chaque page pour le référencement (extrait affiché dans les résultats de recherche)
- ▶ Référencement naturel limité par nature : l'application est derrière une authentification, donc peu indexable au-delà de la page de connexion

Audit Lighthouse — page index.html

PERFORMANCE

100

ACCESSIBILITÉ

96

BEST PRACTICES

100

SEO

100

CAHIER DE TESTS

12 scénarios manuels exécutés contre le serveur réel

Test	Description	Comment / ce que ça vérifie	Résultat
TEST001	Accessibilité du site	30 critères RGAA/WCAG à la main : navigation clavier complète, aria-label sur les boutons, contrastes, label par champ	OK
TEST002	Connexion avec un compte valide	Vérifie que le bon email/mot de passe renvoie un token JWT	OK
TEST003	Connexion avec un compte inexistant	Vérifie que le serveur refuse et renvoie 401, pas d'information révélée	OK
TEST004	Accès à une route protégée sans token	Vérifie que le middleware verifyToken bloque bien avec 401	OK
TEST005	Création d'une tâche sans titre	Vérifie le rejet (400) si le champ obligatoire manque	OK
TEST006	Création d'une tâche valide	Vérifie l'enregistrement réel en base et la réponse 201	OK
TEST007	Liste des colonnes	Vérifie l'ordre par position et la structure de la réponse	OK
TEST008	Déplacement d'une tâche entre colonnes	Vérifie que id_col change bien en base après un PUT	OK
TEST009	Suppression d'une colonne avec une tâche	Vérifie le ON DELETE CASCADE : la tâche disparaît avec la colonne	OK
TEST010	Liste des utilisateurs	Vérifie que le mot de passe n'est jamais présent dans la réponse JSON	OK
TEST011	Inscription d'un nouveau compte	Vérifie le hachage bcrypt et la création en base	OK
TEST012	Connexion avec le compte fraîchement inscrit	Vérifie que le compte créé fonctionne immédiatement	OK

Outils : Swagger UI et DevTools du navigateur. Détail complet dans [cahier_tests_accessibilite.html](#) et [Cahier_de_Tests_Lumina_Pro.html](#).

DÉPLOIEMENT

Fonctionnement en local

LOCAL

- MySQL 8.0.45 + `node server.js`
- Démarrage manuel : `service MySQL80` puis `npm start`

TEST

- Variables dans `.env`
- Comptes de démo créés via `/api/auth/register`

Aujourd'hui, le projet tourne entièrement en local (XAMPP + Node.js).

AMÉLIORATIONS FUTURES

Ce que j'améliorerais en production

SÉCURITÉ

- HTTPS obligatoire
- Rate limiting anti-bruteforce sur Redis (déjà en place en mémoire)

QUALITÉ

- Tests automatisés Jest
- Validations backend renforcées

FONCTIONNALITÉS

- Drag & drop natif
- Docker pour le déploiement

CONCLUSION

Lumina Pro m'a permis de mettre en pratique l'ensemble des compétences du titre DWWM :

- ✓ **Front-end** : HTML/CSS responsive, JavaScript modulaire, Fetch API
- ✓ **Back-end** : API REST Node.js/Express, authentification JWT, hachage bcrypt
- ✓ **Base de données** : conception MCD/MLD/MPD, SQL, relations
- ✓ **Sécurité** : injection SQL, XSS, mots de passe, autorisation par propriétaire

C'est un bon début pour un projet que je compte continuer à faire vivre au-delà de l'examen. Pour qu'il se distingue vraiment, les prochaines étapes seraient : le drag & drop natif pour une expérience plus fluide, des tests automatisés pour sécuriser les évolutions futures, et une vraie mise en production accessible en ligne — plutôt que d'ajouter des fonctionnalités superflues, je préfère consolider ce cœur applicatif avant de l'étendre.



Merci pour votre attention

— Questions ? —



Lumina Pro

TABLEAU KANBAN · APPLICATION WEB SÉCURISÉE

Syrine Ben Hassine

syrine.ben-hassine@laplateforme.io

Merci pour votre attention

— Questions ? —